

NSS FuelTrack — Complete Developer Guide

Version: FINAL v3 | **Prepared by:** Sai (for brother to build/maintain)

TABLE OF CONTENTS

1. [What Is This App?](#)
2. [File Structure](#)
3. [How to Deploy \(Host the App\)](#)
4. [Full Code Breakdown](#)
5. [Key Features Explained](#)
6. [Data Flow & State Management](#)
7. [OTP Security System](#)
8. [Payment Modes](#)
9. [Nozzle & Fuel Configuration](#)
10. [WhatsApp Integration](#)
11. [How to Modify / Extend](#)

12. [Common Bugs & Fixes](#)

13. [Testing Checklist](#)

1. WHAT IS THIS APP?

NSS FuelTrack is a **Progressive Web App (PWA)** – a website that installs on Android/iPhone like a native app.

Purpose: DSMs (Dispensing Staff) at NSS Fuel Station, Nangali, Karnataka use this to:

- Log every fuel transaction (nozzle, litres, payment mode, vehicle)
- Track cash/UPI/credit/digital collections
- Generate shift reports and share via WhatsApp
- Allow the Owner to monitor remotely via WhatsApp alerts

Key constraint: Works 100% OFFLINE. All data lives in the phone's `localStorage`. No server. No database. No backend.

2. FILE STRUCTURE

```
nss-pwa/  
├── index.html      ← The ENTIRE app  
                        (HTML + CSS + JS in one file, ~2800  
                        lines)  
├── manifest.json   ← PWA config  
                        (name, icons, theme colour)  
├── sw.js           ← Service Worker  
                        (makes app work offline)  
├── icon-192.png    ← App icon (home  
                        screen, 192×192)  
└── icon-512.png    ← App icon  
                        (splash screen, 512×512)
```

That's it. 5 files. The entire app lives in
`index.html` .

3. HOW TO DEPLOY

Option A — GitHub Pages (FREE, Recommended)

1. Create a GitHub account at github.com
2. Create a new repository, e.g. `nss-fueltrack`

3. Upload all 5 files from the ZIP
4. Go to **Settings** → **Pages** → **Source** → **main branch** → / (root)
5. Save. In ~2 minutes the app is live at:
`https://YOUR_USERNAME.github.io/nss-fueltrack/`
6. Share that URL with DSMs. On Android Chrome, they tap "**Add to Home Screen**" — it installs like an app.

Option B — Netlify (FREE, easier drag & drop)

1. Go to netlify.com → Sign up free
2. Drag the entire `nss-pwa/` folder onto the Netlify dashboard
3. Get a URL like `https://random-name.netlify.app`
4. Optional: buy a domain and link it

Option C — Local (for testing only)

Run a local server (do NOT just open index.html directly — service workers need HTTP):

```
# Python 3 (if installed)
cd nss-pwa
python -m http.server 8080
# Then open: http://localhost:8080
```

4. FULL CODE BREAKDOWN

manifest.json – PWA Identity

```
{
  "name": "NSS FuelTrack",
  "short_name": "FuelTrack",
  "description": "DSM Payment
Tracking App - NSS Fuel Station,
Nangali",
  "start_url": "./index.html",
  "display": "standalone",           ←
  Hides browser chrome (looks like
  native app)
  "orientation": "portrait",
  "background_color": "#0f172a",    ←
  Splash screen background
  "theme_color": "#f59e0b",         ←
  Phone status bar colour (gold)
```

```
"icons": [...]  
}
```

To change: App name, icon, or colours → edit this file.

sw.js — Service Worker (Offline Support)

```
const CACHE = 'nss-fueltrack-v1';  
const ASSETS = ['./index.html',  
  './manifest.json', './icon-192.png',  
  './icon-512.png'];
```

The service worker:

1. On first load → **caches all 4 files** in the browser
2. On every request → **serves from cache first**, falls back to network
3. On update → deletes old cache versions

IMPORTANT: When you update the app, increment the cache version:

```
const CACHE = 'nss-fueltrack-v2'; //  
bump this on every update
```

Otherwise phones won't download the new version.

index.html – The Entire App

Divided into 5 sections:

Section 1: `<head>` – Meta tags & CSS (~400 lines)

PWA meta tags, BPCL blue/gold colour theme, all component styles.

Key CSS variables:

```
:root {
  --gold: #f59e0b;      /* NSS gold
  */
  --hsd: #3b82f6;      /* Diesel =
  blue */
  --ms: #f59e0b;       /* Petrol =
  gold */
  --card: #111827;     /* Card
  backgrounds */
}
```

BPCL official colours (overridden at bottom of CSS):

```
/* Header is BPCL deep blue */
#hdr { background: linear-
gradient(135deg,#001440,#003087,#0014
40) !important; }
/* DSM name in BPCL gold */
#dsm-name { color: #FFD100
!important; }
```

Section 2: HTML Screens

The app has these **screens** (only one visible at a time):

Screen ID	What the user sees
#setup	Login screen (DSM name + OTP + totalizer readings)
#s-dash	Home dashboard (totals, mode tiles, recent transactions)
#s-entry	New transaction entry form

Screen ID	What the user sees
#s-report	Shift report (mode breakdown + WhatsApp share)
#s-hist	Transaction history
#s-owner	Owner settings (protected by PIN)

Plus **modals** (pop-up overlays):

- #pin-modal — 4-digit PIN keypad
- #otp-modal — OTP entry for transaction edits
- #edit-modal — Edit a transaction (one-time only)
- #ded-modal — Add Loyalty Reward or Expense
- #settlement-overlay — Full-screen Shift Settlement calculator

Section 3: JavaScript — State & Config

App Configuration (hardcoded — change here to modify behaviour):

```

// PAYMENT MODES (order shown in
Entry tab)
const MODES = [
  {id:'cash',      label:'Cash',
  icon:'💰', color:'#22c55e'},
  {id:'upi',       label:'UPI',
  icon:'📱', color:'#3b82f6'},
  {id:'penlabs',   label:'Pine Labs',
  icon:'🇮🇳', color:'#8b5cf6'},
  {id:'otp',       label:'Bharath
OTP/Card',icon:'👤',
color:'#ec4899'},
  {id:'phonepay',  label:'PP EDC',
  icon:'🏧', color:'#f97316'},
  {id:'credit',    label:'Credit',
  icon:'📋', color:'#ef4444'},
  {id:'testing',   label:'Testing',
  icon:'🔧', color:'#6366f1'}, //
hidden from sales
];

// NOZZLE LAYOUT (currently 1 pump
with 4 nozzles: N1,N2=HSD, N3,N4=MS)
const NOZZLES = [
  {id:1, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
  {id:2, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
  {id:3, fuel:'ms',  label:'MS',

```

```
color:'#f59e0b'},
  {id:4, fuel:'ms', label:'MS',
color:'#f59e0b'},
];

// VEHICLE TYPES
const VEHICLES = [
  {id:'bike', emoji:'🏍️',
label:'Bike'},
  {id:'car', emoji:'🚗',
label:'Car'},
  {id:'auto', emoji:'🚚',
label:'Auto'},
  {id:'lorry', emoji:'🚛',
label:'Lorry'},
  {id:'bus', emoji:'🚌',
label:'Bus'},
  {id:'other', emoji:'🚕',
label:'Other'},
];

// TURBO MODE quick-amount buttons
(Rs.)
const QA_AMOUNTS = [100, 200, 500,
1000, 2000];
```

App State Object (everything stored here at runtime):

```

let S = {
  dsmName: '',          // DSM's
name (entered at login)
  shiftType: 'day',      // unused
currently
  startTime: null,       // Date
object when shift started
  openTotl: {            // Opening
totalizer readings
  n1: 0,                 // N1
opening meter
  n2: 0,
  n3: 0,
  n4: 0,
  },
  transactions: [],      // Array of
transaction objects
  selVehicle: 'bike',    //
Currently selected vehicle
  selNozzle: null,       //
Currently selected nozzle ID (1-4)
  selMode: '',           //
Currently selected payment mode ID
  turbo: false,          // Turbo
mode on/off
  selQA: null,           // Selected
quick-amount in turbo
  deductions: [],        // Loyalty

```

```
rewards & expenses  
};
```

5. KEY FEATURES EXPLAINED

5.1 Shift Login Flow

```
DSM types name  
↓  
Taps "Send OTP to Owner"  
↓  
App generates 6-digit OTP → Opens  
WhatsApp to owner's number with OTP  
↓  
Owner receives WhatsApp → tells DSM  
the OTP  
↓  
DSM enters OTP → verified in-app (no  
server)  
↓  
Step 2 appears: Enter 4 opening  
totalizer readings (N1-N4)  
↓  
"Start Shift" button enabled → shift  
begins
```

Owner Direct Login (for owner to log in without going through DSM flow):

- Tap "Owner Direct Login" on setup screen
- Enter 4-digit PIN
- Goes directly to Owner tab

5.2 Transaction Entry

Normal Mode:

1. Select Nozzle (N1–N4)
2. Select Vehicle type
3. Enter Litres → Amount auto-calculates at current rate
4. Select Payment Mode
5. If Credit → enter customer name (required), phone, vehicle number
6. Tap "Save Transaction"

Turbo Mode (for heavy traffic):

- Select Nozzle
- Tap quick amount
(Rs.100/200/500/1000/2000) or type custom

- Litres auto-calculate
- Tap Payment Mode → **auto-saves immediately**
- No vehicle selection, no note – fastest possible entry

5.3 Auto-Amount Calculation

```
function calcAmt() {
  const l =
parseFloat(document.getElementById('l
it-inp').value);
  if (!isNaN(l) && l > 0 &&
S.selNozzle) {
    const n = NOZZLES.find(x => x.id
=== S.selNozzle);
    const rate = n.fuel === 'ms' ?
prices.ms : prices.hsd;
    const calc = (l *
rate).toFixed(2);
    document.getElementById('amt-
inp').value = calc;
  }
}
```

Fuel prices stored in `localStorage` as `nss_prices`. Default: MS=₹102.50,

HSD=₹89.00.

5.4 Live Totalizer (Dashboard)

Shows current meter reading = opening reading + all litres sold through that nozzle this shift:

```
function updRunningTotl() {
  NOZZLES.forEach(n => {
    const added = S.transactions
      .filter(t => t.nozzle === n.id)
      .reduce((s, t) => s + t.liters,
        0);
    const current = (S.openTotl['n' +
      n.id] || 0) + added;
    // displays current in dashboard
    tiles
  });
}
```

5.5 Deductions (Loyalty Rewards & Expenses)

These are amounts deducted from the **cash collection** (not from digital payments):


```
// A deduction object looks like:
{
  type: 'reward',          // 'reward'
or 'expense'
  amount: 500,             // amount in
Rs.
  note: 'KA01AB1234',     // vehicle
number (rewards) or description
(expenses)
  time: '2026-06-09T...' // ISO
timestamp
}
```

They appear in Dashboard, Entry tab, Report tab, History tab, and Settlement calculator — all deducted from Cash.

5.6 Shift Settlement Calculator (Owner only)

Accessed from Owner tab → "Open Settlement Calculator".

Computes:

- **Expected collection** = HSD litres × HSD rate
+ MS litres × MS rate

- **Actual collection** = sum of all payment mode inputs (editable by owner)
 - **Net cash to hand over** = Cash collected – Loyalty rewards – Expenses
 - **Result** = Actual – Deductions – Expected → BALANCED / SHORTAGE / EXCESS
-

6. DATA FLOW & STATE MANAGEMENT

Transaction Object Structure

```
// Every transaction saved looks like this:
{
  id: 'lx7k2m',           //
  unique ID (Date.now().toString(36))
  vehicle: 'Lorry',       //
  vehicle label
  vehicleEmoji: '🚚',
  nozzle: 2,              //
  nozzle ID (1-4)
  fuelId: 'hsd',          // 'hsd'
  or 'ms'
  fuelLabel: 'HSD (Diesel)',
```

```

    liters: 45.23,           //
litres dispensed
    amount: 4025.47,        // Rs.
collected
    paymentMode: 'cash',    //
payment mode ID
    modeLabel: 'Cash',
    rateUsed: 89.00,        // fuel
rate at time of transaction
    note: 'Lorry from Bangalore',
    creditName: '',         // only
for credit mode
    creditPhone: '',
    creditVehicle: '',
    timestamp: Date,        //
JavaScript Date object
    edited: false,          //
becomes true after OTP edit
    editedAt: null,         // ISO
string when edited
    editedBy: null,         // DSM
name who edited
}

```

localStorage Keys

```

nss_shift      → Full shift data
(JSON): dsmName, startTime, openTotl,

```

```
transactions, deductions
nss_prices      → Fuel prices (JSON):
{ms: 102.50, hsd: 89.00}
nss_owner_phone → 10-digit owner
WhatsApp number (string)
nss_pin         → 4-digit owner PIN
(string, default: '0000')
```

Auto-Save

Every time a transaction is saved,

`saveState()` is called:

```
function saveState() {
  const d = {
    v: 2,
    dsmName: S.dsmName,
    startTime:
S.startTime.toISOString(),
    openTotl: S.openTotl,
    deductions: S.deductions,
    transactions:
S.transactions.map(t => ({
      ...t,
      timestamp:
t.timestamp.toISOString() // convert
Date to string for JSON
    })),
  };
}
```

```
localStorage.setItem('nss_shift',  
JSON.stringify(d));  
}
```

If the page refreshes/crashes, **all data is safe**.
On next open, the restore banner appears.

7. OTP SECURITY SYSTEM

The app uses **WhatsApp OTP** (no SMS, no server). OTPs are generated in-app, sent via WhatsApp URL.

OTPs used for:

Action	Who receives OTP	Where sent
DSM Shift Login	Owner	Owner's WhatsApp
Change Fuel Prices	Owner	Owner's WhatsApp

Action	Who receives OTP	Where sent
Change Owner PIN	Owner	Owner's WhatsApp
Change Owner Phone	Owner (current number)	Current WhatsApp
Edit a Transaction	Owner	Owner's WhatsApp

OTP Logic (same pattern everywhere):

```
// 1. Generate OTP
_otp = Math.floor(100000 +
Math.random() * 900000).toString();
// 6-digit
_otpExpiry = Date.now() + 5 * 60 *
1000; // expires in 5 minutes

// 2. Open WhatsApp with OTP in
message
const msg = '*NSS FuelTrack OTP*
...\nOTP: *' + _otp + *';
window.open('https://wa.me/91' +
```

```
phone + '?text=' +  
encodeURIComponent(msg), '_blank');  
  
// 3. Verify  
function verifyOtp() {  
    if (!_otp) { /* expired */ return;  
}  
    if (Date.now() > _otpExpiry) { /*  
expired */ _otp = null; return; }  
    if (entered !== _otp) { /* wrong */  
return; }  
    // CORRECT → do the action  
    _otp = null; // invalidate  
}
```

Transaction Edit Rules:

- Only **one edit per transaction** (after edit, `tx.edited = true`)
- Requires owner OTP
- Owner sees exactly what is being edited in the WhatsApp message
- Edit is logged (who edited, when)

8. PAYMENT MODES

Mode ID	Label	Icon	Included in Sales?
cash	Cash		YES
upi	UPI		YES
penlabs	Pine Labs		YES
otp	Bharath OTP/Card		YES
phonepay	PP EDC		YES
credit	Credit		YES
testing	Testing		NO (excluded from actual sales)

Testing mode: Litres are counted in totalizer (fuel physically dispensed) but amount is excluded from financial reports. Used for machine calibration, own vehicle fills, etc.

9. NOZZLE & FUEL CONFIGURATION

Current configuration: 1 pump, 4 nozzles:

- N1 = HSD (Diesel), N2 = HSD (Diesel)
- N3 = MS (Petrol), N4 = MS (Petrol)

To add more nozzles (e.g., for 3-pump expansion to N1–N12):

```
// In the NOZZLES config at top of JS
section:
const NOZZLES = [
  // Pump 1
  {id:1, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
  {id:2, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
  {id:3, fuel:'ms', label:'MS',
color:'#f59e0b'},
  {id:4, fuel:'ms', label:'MS',
color:'#f59e0b'},
  // Pump 2
  {id:5, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
  {id:6, fuel:'hsd', label:'HSD',
color:'#3b82f6'},
```

```
{id:7, fuel:'ms', label:'MS',  
color:'#f59e0b'},  
  {id:8, fuel:'ms', label:'MS',  
color:'#f59e0b'},  
  // ... etc up to N12  
];
```

Also update the **Setup screen** (opening readings grid) and **Dashboard** (live totalizer tiles) to match. Currently hardcoded for 4 nozzles in HTML.

10. WHATSAPP INTEGRATION

The app uses **WhatsApp URL scheme** — no WhatsApp API, no cost, no approval needed:

```
https://wa.me/91XXXXXXXXXX?  
text=MESSAGE
```

This opens WhatsApp on the device with the pre-filled message. The user taps Send.

Scheduled Auto-Reports (1 PM & 8 PM)

```

function checkSchedule() {
  if (!S.startTime) return;
  const now = new Date();
  const h = now.getHours(), m =
now.getMinutes();
  const today = now.toDateString();

  // 1:00 PM
  if (h === 13 && m === 0 &&
scheduledSentToday.slot1pm !== today)
{
  scheduledSentToday.slot1pm =
today;
  sendScheduledWA('1:00 PM
UPDATE');
}
  // 8:00 PM
  if (h === 20 && m === 0 &&
scheduledSentToday.slot8pm !== today)
{
  scheduledSentToday.slot8pm =
today;
  sendScheduledWA('8:00 PM
UPDATE');
}
}
setInterval(checkSchedule, 30000); //
checks every 30 seconds

```

Limitation: App must be open (in foreground or background). Closed app = no auto-send. This is a browser limitation — true background scheduling needs a native app or server.

11. HOW TO MODIFY / EXTEND

Change Fuel Prices (Default)

```
let prices = { ms: 102.50, hsd: 89.00
}; // line ~200 in JS section
```

Change Scheduled Report Times

```
// Currently 1 PM (hour=13) and 8 PM (hour=20)
if (h === 13 && m === 0 ...) //
change 13 to desired hour (24-hr format)
if (h === 20 && m === 0 ...) //
change 20 to desired hour
```

Add a New Payment Mode

1. Add to `MODES` array:

```
{id:'newmode', label:'My New Mode',  
icon:'NEW', color:'#hex'}
```

2. The pay-grid and all reports auto-update (they loop through MODES).
3. If it needs special fields (like credit does), add HTML in the Entry screen and handle in `saveTx()`.

Change the Default PIN

```
function getPin() { return  
localStorage.getItem('nss_pin') ||  
'0000'; }  
//  
↑ change this
```

Change App Colours (BPCL theme)

Find this block near bottom of `<style>`:

```
#hdr { background: linear-  
gradient(135deg,#001440,#003087,#0014  
40) !important; }  
/* #001440 = BPCL navy, #003087 =  
BPCL blue, #FFD100 = BPCL gold */
```

Add Another Screen

1. Add HTML: `<div id="s-newscreen" class="screen"> ... </div>`
 2. Add nav button: `<button class="nb" id="nbX" onclick="goTo('newscreen',this)">... </button>`
 3. The `goTo()` function handles hiding/showing automatically.
-

12. COMMON BUGS & FIXES

Bug: App not updating after deploying new version

Fix: Increment cache version in `sw.js`:

```
const CACHE = 'nss-fueltrack-v2'; //
was v1, change to v2, v3, etc.
```

Alternatively, users can go to Chrome Settings
→ Site Settings → Clear Data for the site.

Bug: WhatsApp not opening / wrong number

Fix: Check `nss_owner_phone` in localStorage:

```
// In browser console:  
localStorage.getItem('nss_owner_phone'  
)  
// Should return 10-digit number  
without +91
```

If empty, owner must set it in Owner tab first.

Bug: OTP expired immediately

Fix: OTP is valid 5 minutes for most, 10 minutes for login OTP. The check uses `Date.now()`. If the phone's clock is wrong, this breaks. Check device time.

Bug: Transactions lost after page refresh

This should NOT happen due to auto-save. If it does:

1. Check if localStorage is available (private/incognito mode disables it)

2. Check storage quota – very old phones may have <5MB localStorage
3. The `saveState()` has a try/catch that silently fails – add `console.error` to debug.

Bug: Amount not calculating

Fix: A nozzle must be selected BEFORE entering litres. Nozzle selection sets `S.selNozzle` which `calcAmt()` reads.

Bug: "Save Transaction" button stays disabled

Fix: All three must be selected:

- Nozzle (`S.selNozzle`)
- Litres & Amount (non-empty input fields)
- Payment mode (`S.selMode`)
- For credit: Customer name (`cr-name` field)

13. TESTING CHECKLIST

Before releasing any update, test these flows:

Setup:

- [] DSM login with OTP (check WhatsApp opens to correct number)
- [] Opening totalizer readings entered and validated
- [] Shift starts, header shows DSM name
- [] Owner direct login with PIN works

Entry:

- [] Normal mode: select nozzle → enter litres → amount auto-calculates
- [] Turbo mode: select nozzle → select quick amount → tap payment → auto-saves
- [] Credit mode: customer name required before save
- [] Testing mode: transaction saves but shows in testing section only
- [] Save flash appears after saving
- [] Transaction count increments in header

Dashboard:

- ☐ Grand total updates after each transaction
- ☐ Mode tiles (Cash/UPI/etc.) show correct amounts
- ☐ Live totalizer shows opening + sold = current reading
- ☐ Loyalty reward adds and shows in deduction summary
- ☐ Expense adds and shows in deduction summary

Report:

- ☐ Closing readings auto-update (no manual entry needed)
- ☐ Match/Mismatch indicator works
- ☐ WhatsApp share opens with complete message
- ☐ PDF/print view shows all transactions

History:

- ☐ All transactions listed newest-first
- ☐ Delete requires owner PIN
- ☐ Edit requires owner OTP → one-time only, locked after

Owner Tab:

- ☐ PIN gate works (default 0000)
- ☐ Change prices requires OTP
- ☐ Change PIN requires OTP
- ☐ Change WhatsApp number requires OTP to current number
- ☐ Settlement calculator computes correctly
- ☐ Settlement WhatsApp share and PDF download work

PWA:

- ☐ Install banner appears on first visit (Android Chrome)
- ☐ App installs to home screen
- ☐ App works offline after first load
- ☐ Service worker updates on cache version change

QUICK REFERENCE: Function Map

Function	What it does
<code>startShift()</code>	Launches the app after login/readings
<code>launchApp()</code>	Shows app UI, starts clock, builds entry UI
<code>saveTx()</code>	Saves a transaction (normal mode)
<code>saveTxTurbo()</code>	Saves a transaction (turbo mode)
<code>calcAmt()</code>	Auto-calculates amount from litres $\times$ rate
<code>updDash()</code>	Refreshes entire dashboard
<code>updReport()</code>	Refreshes shift report
<code>updHist()</code>	Refreshes history tab
<code>updDed()</code>	Refreshes deductions in all views
<code>updRunningTotl()</code>	Updates live meter readings on dashboard

Function	What it does
<code>updClosingTotl()</code>	Updates auto-closing readings in report
<code>goTo(name, btn)</code>	Navigate between screens
<code>showPin(t,s,cb)</code>	Show PIN modal, run callback on success
<code>requestEdit(txId)</code>	Start OTP flow for transaction edit
<code>openSettlement()</code>	Open shift settlement overlay
<code>calcSettlement()</code>	Compute shortage/excess/balance
<code>doWhatsApp()</code>	Quick WhatsApp shift summary
<code>doShareReport()</code>	Full report via Web Share API
<code>doPDF()</code>	Print view (browser print dialog)

Function	What it does
<code>saveState()</code>	Save all shift data to localStorage
<code>loadSaved()</code>	Load saved shift from localStorage
<code>fmt(n)</code>	Format number as ₹ currency string
<code>fmtTime(d)</code>	Format Date as "10:30 AM"
<code>fmtDate(d)</code>	Format Date as "09 Jun 2026 10:30 AM"

CONTACT / OWNERSHIP

- **Station:** NSS Fuel Station, Nangali, Mulabagal Taluk, Kolar Dist, Karnataka
 - **Proprietor:** KM Manjulamma
 - **App built by:** Sai (MBA, accounts manager)
 - **Version:** FINAL v3, June 2026
-

This guide is meant for the technical collaborator (brother) to understand, host, maintain and extend the app.